

A Critical Code Review of the Web Witchcraft and Wizardry Project

Edmund Mulligan

April 9, 2026

Abstract

Abstract goes here.

1 Introduction

This essay reviews the codebase of the Web Witchcraft and Wizardry project, a web-based interactive learning platform designed to teach users about web development concepts through a magical theme. The review covers the architecture and design of the project, as well as an analysis of the HTML, CSS, and JavaScript code, with an emphasis on the latter. Reflections on the development process and potential areas for improvement are also discussed.

2 Architecture and Design

The design of the appearance and layout of the website was documented in the previous essay (Mulligan, 2026), so will not be covered here, but the code has also been designed so as to be maintainable and extensible, with a modular structure that separates concerns (Dijkstra, 1982). This is a front-end focused project, so the codebase is primarily composed of HTML, CSS, and JavaScript files, with a clear separation between structure, presentation, and behaviour. The project also utilises a build process to allow automated testing, as well as a version control system to manage changes and collaborate with other developers (should there be any in the future).

Several constraints have been imposed on the project, partly to comply with the academic requirements of this course, but also to keep the project focused and manageable.

- Use only vanilla JavaScript, HTML, and CSS, without any external libraries or frameworks.
- No use of generative AI or Large Language Models (LLMs) in the development process, to ensure that the code is entirely original and not influenced by AI-generated content.
- No server side code or databases, as the project is focused on front-end development and client-side interactivity.
- Phases 1 and 2 of the project had fixed deadlines, although phase 3 (after submission of this assignment) is more flexible.
- Ensure that the website is fully accessible and responsive, adhering to best practices in web development.
- The project is developed by a single individual (myself).

3 HTML and CSS

Since the focus of this assignment is Javascript, the HTML and CSS code will not be analysed in depth, but some general observations can be made. The HTML code is well-structured and semantic, with appropriate use of headings, lists, and other elements to create a clear and logical document structure. The CSS code is organised into separate files for layout and components, with consistent naming conventions and a clear separation of concerns. The use of CSS variables allows for easy theming and customization of the website's appearance. The CSS has been completely refactored since the last assignment, to make it more maintainable and efficient. This includes using layers to control the cascade and specificity of styles, as well as removing redundant and unused styles. Nested selectors have been used where possible to improve readability and reduce repetition. At most three CSS files need to be loaded on any page (globals.css, main.css and a page specific CSS file if necessary), which makes it less likely a specific CSS file will be accidentally missed when developing new pages. The CSS code has been heavily influenced by tutorials from Kevin Powell (Powell, 2026b) and his paid course (Powell, 2026a), which have provided valuable insights into modern CSS techniques and best practices.

4 Javascript

5 Reflections

Conclusion

Word count

Word count: 516 words (excluding references)

References

- Edsger W. Dijkstra. On the role of scientific thought. In *Selected Writings on Computing: A Personal Perspective*, pages 60–66. Springer, New York, 1982. Originally published in 1974. Introduces the principle of separation of concerns.
- Edmund Mulligan. A critical reflection on the design of the web witchcraft and wizardry project, 2026.
- Kevin Powell. Css demystified. Online course, 2026a. URL <https://courses.thecascade.dev/course/css-demystified/>. Paid course. Accessed: April 2026.
- Kevin Powell. Kevin powell youtube channel, 2026b. URL <https://www.youtube.com/@KevinPowell>. Accessed April 2026.

Appendices

A Javascript Assessment Criteria

Here are examples of Javascript concepts used in the codebase to allow assessors to easily identify where key features are implemented. This is not an exhaustive list of all Javascript concepts used in the codebase, but rather a quick guide to where an instance of each key feature can be found. Assessors are, of course, free to review the codebase in its entirety to gain a deeper understanding of the implementation and design choices made throughout the project.

A.1 Variables and Functions — `animatePortrait.js`

`setRandomPortraitPosition (container)` is a method (function) that uses variables to store the position of the portrait and the dimensions of the container. It also calls JavaScript's built-in `Math.random ()` function to generate random positions for the portrait.

A.2 Objects and Arrays — `animatePortrait.js`

`PortraitAnimator` class is instantiated as an object and uses an array to store the portrait images that are animated.

A.3 Conditional Logic and Loops — `animateWand.js`

Uses a `foreach` loop to iterate over path elements in `svg` base element and a conditional to deal with the possibility that the `svg` file does not contain a base `svg` element and to determine if a path element should be animated with a sparkle.

A.4 DOM Manipulation — `animatePortrait.js`

The DOM is manipulated in method `setPortraitImage (container, image)` to change the source of the portrait image and update the alt text.

A.5 Interactivity — `toggleSection.js`

`toggleSectionCollapse (sectionId, event)` is a method that toggles the collapse state of a section in response to user triggered events (e.g. clicking a button). It uses event listeners to detect user interactions and updates the DOM accordingly to show or hide the content of the section.

A.6 Data Retrieval and Storage — `localStorage.js`

The `LocalStorage` class provides methods for storing and retrieving encrypted data from the browser's local storage. This allows the application to persist user preferences or other data across sessions.